Jev-Backed Function Calling: Dual Verification with Bounded Repair

Anonymous Author(s)

Abstract

Function-calling language models must select an appropriate function and instantiate its arguments before software acts on a user's request. Syntactically valid calls can still fail either requirement. We present Jev-Backed Function Calling, an inference-time protocol that inserts TypeSafe's Jev decision model between call generation and dispatch. Jev evaluates function selection and argument correctness as two typed judgments in one request. A candidate passes only when both judgments meet their thresholds; otherwise, structured feedback conditions a revised proposal from the same generator. The protocol permits at most three repairs after the initial proposal and specifies behavior when this budget is exhausted. We provide a reproducible evaluation package for 3,051 examples from seven single-turn categories of the Berkeley Function Calling Leaderboard, with disjoint development and test splits, official Python call checking, comparison methods, and quality–cost diagnostics. This manuscript completes the method and experimental protocol; empirical results remain pending. The study is designed to determine whether external dual verification improves final call correctness beyond bounded self-revision and whether any gains justify its overhead.

1 Introduction

A function-calling system receives a user request and function catalog, asks a language model to propose a call, and dispatches it. The generator must choose an operation and assign its arguments. A travel request might select the wrong booking function or swap departure and destination; either error can satisfy a JSON schema.

Improvements in tool-use training and evaluation have made this interface increasingly practical [2, 3, 1]. However, schema conformance establishes structural validity, not agreement with user intent. The gap motivates an explicit check before dispatch. Such a check should assess the selected operation against all available alternatives and assess the proposed values against the request and the selected signature. A failure in either component is sufficient to make the call unsuitable.

We investigate a particular implementation of this idea using Jev, TypeSafe's System One model [12]. Jev accepts a shared state and typed questions and returns structured decisions rather than free-form generated explanations. We use two yes/no probability questions: whether the function selection is appropriate and whether the arguments are correct. Their conjunction controls early acceptance. Failed checks produce compact, deterministic feedback for the generator, which proposes a replacement. Each replacement is checked afresh; neither a previously correct function nor previously accepted arguments are presumed to remain correct after revision.

Verification and feedback-conditioned correction have substantial precedent. ToolVerifier checks both tool selection and parameter generation [9], and ToolCritic provides diagnostic feedback for revising tool-use responses [10]. TypeSafe also documents direct function routing and an extraction-verification cascade [13, 14]. Our contribution is therefore the concrete Jev-backed protocol and an auditable evaluation of its tradeoffs, not a claim that tool verification or iterative repair is new.

The implementation requires both checks to pass on the same candidate, limits repair to three rounds, and distinguishes correctness from acceptance and dispatch. We compare direct calling, schema repair, self-revision, and single-check ablations on a frozen BFCL subset, measuring corrected and damaged predictions alongside latency and cost.

2 Related Work

Tool use and benchmarks. Gorilla studies API-call generation [2], and ToolLLM introduces tool-use training and evaluation [3]. BFCL evaluates function calling across broader settings [1]; we select its single-turn Python categories to isolate pre-dispatch decisions. API-Bank provides runnable dialogues [4], while StableToolBench addresses changing external APIs through caching and simulation [5]. These motivate future evaluation beyond our BFCL subset.

Feedback and verification. Self-Refine [6], Reflexion [7], and CRITIC [8] use self-generated, linguistic, and tool-derived feedback, respectively. Intrinsic self-correction can also degrade correct answers [11], motivating our correction/regression measurements. ToolVerifier separates tool selection and parameter verification using contrastive questions [9]; ToolCritic diagnoses tool-use errors and feeds them back to the generator [10]. Our frozen external API evaluates complete proposals under a bounded repair policy.

Jev as a judge. Li et al.'s *JEV-as-a-Judge* [15] evaluates finished outputs with typed verdict probabilities and escalates uncertain verdicts to an independent stronger judge. Its frozen cascade reduces evaluation cost while preserving strong-judge accuracy on selected workloads, but difficult derivations, misleading style, and reference-free prose expose

limits. The study explicitly leaves in-agent tool and action selection untested. We instead check proposed function calls and return failed checks to the same generator for revision. TypeSafe's direct closed-set function routing and extraction-verification cascade provide further precedents [13, 14]. Our study measures this specific integration rather than claiming verification or routing as new.

3 Method

3.1 Proposal and Verification

Let x denote the user request, $\mathscr{F}$ the complete supplied function catalog, and G a configured function-calling language model. An initial proposal is

$$c_0 = G(x, \mathscr{F}), \qquad c_t = (f_t, a_t), \tag{1}$$

where f_t identifies a function and a_t its argument object. The representation also permits an empty call for requests that should not invoke a function. The prepared benchmark excludes parallel and multi-turn execution, so the principal task is selecting at most one call from the catalog. An extra call is a prediction error rather than an opportunity to execute arbitrary code.

Deterministic validation checks parsing, function existence, required fields, and declared types. Let $S(c_t)$ indicate whether these checks pass. This inexpensive check is kept separate from Jev so that the schema-only comparison can measure how much correction is attributable to structure alone. Normalization maps benchmark types and function names to the provider's accepted interface while retaining original schemas and names for scoring. The same normalized catalog is supplied to every method.

For every well-formed, parseable candidate, including one that fails schema validation, we construct a state containing x, $\mathscr{F}$, and c_t. It contains no reference answer or correctness label. One Jev request evaluates two `Noul` questions, each oriented so a larger value means a stronger positive judgment:

$$p_f^{(t)} = J_f(x, \mathscr{F}, c_t), \tag{2}$$

$$p_a^{(t)} = J_a(x, \mathscr{F}, c_t). \tag{3}$$

The function question checks whether the proposed operation satisfies the request, including whether abstention is appropriate. The argument question checks whether required values are present, supported by the request, assigned to the correct fields, and consistent with the selected signature. Empty calls require a no-call judgment; they are not accepted merely because there are no arguments to inspect. The questions are distinct judgments sharing a context, not statistically independent measurements.

3.2 Conjunction and Bounded Repair

With thresholds τ_f and τ_a, candidate acceptance is

$$A_t = S(c_t)\,\mathbf{1}[p_f^{(t)} \geq \tau_f]\,\mathbf{1}[p_a^{(t)} \geq \tau_a]. \tag{4}$$

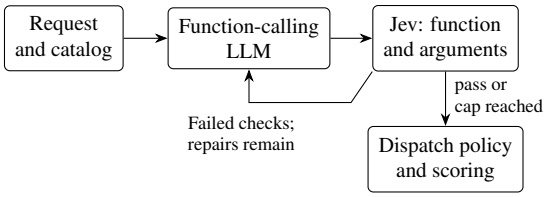


Figure 1: Both checks evaluate the same proposal. The budget permits at most three feedback-conditioned repairs. Dispatch after exhaustion is governed explicitly, independently of acceptance.

The default is $\tau_f = \tau_a = 0.8$, an untuned starting value rather than an empirically justified optimum. Optional calibration uses development data only. Unlike routing by maximum verdict probability [15], a confident negative judgment rejects a call; thresholds must be validated for our Noul interface. Both tests must pass on the same candidate. If either fails, the generator receives the previous proposal and feedback containing the failed criterion, the two probabilities, the thresholds, and a request to reconsider the call using the original catalog and user request. This feedback reports Jev's judgments; it does not invent a natural-language explanation that Jev did not return.

For repair budget $r \in \{0,1,2,3\}$,

$$c_{t+1} = G(x, \mathscr{F}, H_t), \quad 0 \leq t < r, \tag{5}$$

until a candidate passes or the budget is exhausted. Here H_t contains all proposals and feedback through round t, not only the most recent pair. The largest budget therefore permits four generator proposals, not three. Every revised candidate is validated and judged anew. A structural failure overrides acceptance even if both Jev judgments pass. Provider failures are recorded separately and halt the affected request after bounded transport retries; they are not converted into fabricated probability values or negative model judgments.

3.3 Dispatch and Reproducible Traces

The primary `execute_last` policy submits the final proposal to benchmark scoring after budget exhaustion even when Jev rejects it or schema validation fails; malformed proposals score as incorrect. This implements bounded correction followed by completion and prevents acceptance from being mistaken for successful repair. An optional `abstain` policy withholds rejected candidates. It is reported separately because withholding calls changes coverage rather than necessarily correcting them. No benchmark-generated call is dispatched to a live external service: the current study checks call correctness in a controlled evaluator. Thus "execute" denotes the dispatch boundary in the proposed system; empirical claims remain limited to the implemented evaluation.

Each trace saves proposals, structural validation, Jev outputs, feedback, acceptance, stopping reason, returned model versions, token usage, and timings. Reference labels are attached only by offline scoring and cannot flow back into generation or verification. API credentials are read from environment variables and excluded from saved traces. Reusing an

Category	All	Test	Direct	Joint
Simple Python	400	360	–	–
Multiple	200	180	–	–
Irrelevance	240	216	–	–
Live simple	258	233	–	–
Live multiple	1053	948	–	–
Live irrelevance	884	796	–	–
Live relevance[†]	16	15	–	–
Total	3051	2748	–	–

Table 1: Prepared BFCL data and test accuracy (%). Counts are data statistics. [†]Live relevance measures call presence only; total accuracy excludes these 15 cases. – denotes an unrun experiment.

initial proposal across comparison methods preserves a paired starting point; each method is nevertheless charged that proposal's nominal usage and latency for logical per-request accounting.

4 Experiments

4.1 Dataset and Evaluation Scope

We prepare 3,051 BFCL examples from seven categories at repository revision `6ea57973c7a6`[1]. Table 1 records the downloaded category sizes and the planned held-out evaluation. Within each category, examples are ranked by the SHA-256 hash of `seed:id`. The first $\max(1, \lfloor 0.1 N_k \rfloor)$ form development data. Seed 1729 yields 303 development and 2,748 test examples. Model results are not yet available.

The original Python AST checker determines acceptable function calls and argument values for answer-bearing categories. Irrelevance categories assess the required absence of a call. Live relevance supplies call-presence supervision only: it cannot establish that a particular function or its arguments are correct and is excluded from the primary aggregate and corresponding verifier calibration. The main aggregate therefore contains 2,733 test examples per replicate. We report per-category accuracy alongside the micro-average because the larger live categories otherwise dominate it.

BFCL's "multiple" categories provide several candidate functions, not parallel-call execution. The package excludes parallel calls, multi-turn state, web search, memory, and live service execution. The word "live" names a dataset category, not a network request performed by this experiment. Official AST equivalence tests a predicted call against annotated alternatives; it does not demonstrate real-world execution success, authorization, or successful completion of a larger user task.

4.2 Models and Comparison Methods

The default generator is `gpt-5.4-mini-2026-03-17` with reasoning effort `none`; the external checker is the stable, version-pinned `jev-1.13.0`. Both are configurable. We plan three run replicates with the same split and fixed prompts; provider sampling and infrastructure need not be deterministic even when configuration is fixed. The shared initial proposal is sampled once per example and replicate, then reused across methods. Development examples are used for any threshold or prompt selection, followed by a frozen test configuration. Unchanged default thresholds must be labeled untuned.

Method	Acc. (%)	Repairs	Sec.	USD/1k
Direct	–	–	–	–
Schema repair	–	–	–	–
Self-refine	–	–	–	–
Jev: function	–	–	–	–
Jev: arguments	–	–	–	–
Jev: joint	–	–	–	–

Table 2: Primary comparison on the held-out BFCL subset. Accuracy excludes the call-presence-only live relevance category. – denotes an experiment not yet run; no empirical values are imputed.

We compare six methods. *Direct* uses the initial proposal. *Schema-only* repairs deterministic validation failures. *Self-refine* asks the generator to reconsider its own proposal under a fixed revision instruction. This is a generic self-revision control inspired by Self-Refine, not a reproduction of its full task-specific prompting [6]. *Jev-function* and *Jev-arguments* compute both Jev outputs in the same request but use only the named check for gating and feedback. *Jev-joint* uses both. Retaining the same Jev request shape makes the single-check ablations primarily about the gate and feedback, rather than an unrelated change in batching.

All Jev variants retain the deterministic schema floor; a single-check ablation receives only its selected semantic result plus structural errors. All repair methods use the same maximum of three repairs in the principal comparison. This matches the proposal ceiling, not realized tokens, latency, cost, or stopping time. We derive joint-check budgets $r \in \{0,1,2,3\}$ by truncating the same logged three-repair trajectory. The cap is not included in either model's prompt, and no function executes during repair, so these prefixes evaluate alternative stopping policies on identical candidates without additional API calls. Abstention is likewise replayed from the same traces. Usage and latency are accumulated only through the selected stopping point. A zero-repair verifier can change acceptance but cannot improve the initial candidate under the primary completion policy.

4.3 Metrics, Costs, and Statistical Analysis

Final candidate correctness is the primary measure, evaluated independently of the verifier's acceptance. Selection correctness and argument correctness conditional on correct selection localize errors, with their denominators reported. Correctness-conditioned verifier diagnostics exclude live relevance. Let C_i be candidate correctness and A_i the gate decision on a scored candidate. Coverage is $P(A_i = 1)$ and ac-

[1]Full revision: `6ea57973c7a6097fd7c5915698c54c17c5b1b6c8`. The released manifest records source URLs, hashes, normalization, and example IDs.

Joint-check diagnostic	r=0	1	2	3
Final correctness (%)	–	–	–	–
Acceptance coverage (%)	–	–	–	–
Accepted precision (%)	–	–	–	–
False acceptance (%)	–	–	–	–
False rejection (%)	–	–	–	–
Correction (%)	–	–	–	–
Regression (%)	–	–	–	–
Cap exhaustion (%)	–	–	–	–
Mean proposals	–	–	–	–
p95 latency (s)	–	–	–	–

Table 3: Budget and verifier diagnostics; – denotes an unrun experiment. False acceptance conditions on incorrect candidates. Correction and regression are fractions of all scored requests.

cepted precision is $P(C_i = 1 \mid A_i = 1)$. We distinguish false acceptance $P(A_i = 1 \mid C_i = 0)$ from selective risk $P(C_i = 0 \mid A_i = 1)$; the latter complements accepted precision. False rejection is $P(A_i = 0 \mid C_i = 1)$. Undefined conditional metrics are reported as unavailable rather than zero.

Correction and regression track initially wrong candidates becoming correct and initially correct candidates becoming wrong. They are reported with explicit denominators, together with mean repairs, cap exhaustion, parse/schema failures, and API error events. Acceptance and dispatch rates are distinct under `execute_last`. For abstention runs, accuracy on accepted candidates must be accompanied by coverage and correct dispatches per test request. Brier scores for each probability are optional diagnostics on candidates with appropriate labels; results on all visited candidates are distinguished from results on the fixed initial-candidate pool to expose selection effects.

Efficiency reporting includes logical end-to-end latency, p95 latency, generator and Jev request counts, token usage, and estimated cost per 1,000 requests. Shared initial generations are charged to each method's logical usage; observed wall time is saved separately. These usage-based estimates do not reconstruct provider bills or unreported usage from failed requests. Missing provider usage makes cost accounting incomplete rather than implying zero cost. The starting price configuration, dated October 15, 2026, is \$0.75/\$0.075/\$4.50 per million uncached input/cached input/output generator tokens and \$0.042 per million Jev input tokens, with no Jev output charge. Prices are configurable accounting inputs rather than permanent model properties.

For the primary direct-versus-joint difference, we use a paired bootstrap with 2,000 resamples and seed 1729. We first average paired differences across replicates for each example and then resample example IDs, avoiding treatment of replicates as independent test items. Reported intervals are 95% percentile intervals; per-replicate scores accompany them. Tables 2–3 are reporting templates. Their empty cells must not be interpreted as zero accuracy or evidence that the method works. Incomplete or limited smoke runs cannot populate the main table.

4.4 Threats to Validity and Planned Interpretation

A verifier can share the generator's misunderstandings or accept a plausible but unsupported value. Asking two questions does not remove correlated error, and repair can optimize a candidate toward the verifier's preferences rather than the reference task. Repeated testing also changes the candidate distribution, so a threshold's development behavior may not persist over repair rounds. We therefore examine corrections, regressions, false acceptance, and budget-dependent coverage jointly, rather than treating a rising acceptance rate as improvement.

The study is limited to a single generator/verifier pair by default, English-heavy benchmark requests, static schemas, and publicly available examples that may have appeared in model training. Function-name normalization and benchmark annotation conventions can also influence scores. Four documented upstream annotation inconsistencies remain unchanged in the primary denominator. Replication with other generators, independent tool domains, executable tasks, and updated Jev versions is necessary before broad generalization. API outages, incomplete runs, and version changes are operational limitations to disclose, not observations to silently remove. Any paper-level conclusion must follow the measured outcomes, including a null or negative result.

5 Conclusion

Jev-Backed Function Calling checks function selection and arguments before dispatch and feeds failed checks back to the generator for at most three repairs. The evaluation separates structural repair, self-revision, single-check validation, and joint verification, measuring correctness independently of acceptance and cost. The implementation and reporting protocol are ready; empirical effectiveness remains open until the planned runs establish whether corrections outweigh regressions and overhead.

Acknowledgements

This work was created as part of the No-More-Slops group activities at Pseudo Lab.

References

[1] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In *International Conference on Machine Learning*, PMLR 267, pp. 48371–48392, 2025. https://proceedings.mlr.press/v267/patil25a.html.

[2] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large Language Model Connected with Massive APIs. In *Advances in Neural Information Processing Systems*, 37:126544–126565, 2024. https://arxiv.org/abs/2305.15334.

[3] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *International Conference on Learning Representations*, 2024. https://arxiv.org/abs/2307.16789.

[4] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Empirical Methods in Natural Language Processing*, pp. 3102–3116, 2023. https://aclanthology.org/2023.emnlp-main.187/.

[5] Zhicheng Guo, Sijie Cheng, Hao Wang, Shihao Liang, Yujia Qin, Peng Li, Zhiyuan Liu, Maosong Sun, and Yang Liu. StableToolBench: Towards Stable Large-Scale Benchmarking on Tool Learning of Large Language Models. In *Findings of ACL*, pp. 11143–11156, 2024. https://aclanthology.org/2024.findings-acl.664/.

[6] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, 36:46534–46594, 2023. https://arxiv.org/abs/2303.17651.

[7] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, 36:8634–8652, 2023. https://arxiv.org/abs/2303.11366.

[8] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. In *International Conference on Learning Representations*, 2024. https://arxiv.org/abs/2305.11738.

[9] Dheeraj Mekala, Jason Weston, Jack Lanchantin, Roberta Raileanu, Maria Lomeli, Jingbo Shang, and Jane Dwivedi-Yu. ToolVerifier: Generalization to New Tools via Self-Verification. *arXiv:2402.14158*, 2024. https://arxiv.org/abs/2402.14158.

[10] Hassan Hamad, Yingru Xu, Liang Zhao, Wenbo Yan, and Narendra Gyanchandani. ToolCritic: Detecting and Correcting Tool-Use Errors in Dialogue Systems. *arXiv:2510.17052*, 2025. https://arxiv.org/abs/2510.17052.

[11] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large Language Models Cannot Self-Correct Reasoning Yet. In *International Conference on Learning Representations*, 2024. https://arxiv.org/abs/2310.01798.

[12] TypeSafe. Models: Jev and the System One API. Technical documentation, 2026. Accessed October 15, 2026. https://docs.typesafe.ai/models.

[13] TypeSafe. Function Calling. TypeSafe cookbook, 2026. Accessed October 15, 2026. https://docs.typesafe.ai/cookbooks/function_calling.

[14] TypeSafe. SDE Cascade. TypeSafe cookbook, 2026. Accessed October 15, 2026. https://docs.typesafe.ai/cookbooks/sde_cascade.

[15] Yubo Li, Yidi Miao, Ramayya Krishnan, and Rema Padman. JEV-as-a-Judge: Accept When Confident, Escalate When Unsure. *arXiv:2609.26550*, version 5, October 9, 2026. https://arxiv.org/abs/2609.26550v5.